



Inspiring Excellence

Department of Computer Science and Engineering

Course: CSE447: Cryptography and Cryptanalysis

Semester: Spring 2026

Project Report

Title:

Submitted To: [Instructor Name]

Group No: e.g., 01

Section: e.g., 01

Submission Date: [DD Month YYYY]

Group Members

No.	Full Name	Student ID
1	[Member 1 Full Name]	[Member 1 Student ID]
2	[Member 2 Full Name]	[Member 2 Student ID]
3	[Member 3 Full Name]	[Member 3 Student ID]

Table of Contents

1. Introduction and System Overview	3
2. Login and Registration Module	4
3. User Data Encryption and Decryption	5
4. Password Hashing and Salting	6
5. Two-Factor Authentication (2FA)	7
6. Key Management Module	8
7. Post and Profile Management	9
8. Data Storage Security	10
9. Message Authentication Code (MAC)	11
10. Role-Based Access Control (RBAC)	12
11. Secure Session Management	13
12. GitHub Repository and Project Structure	14
13. Conclusion	15

1. Introduction and System Overview

This report documents the design, implementation, and security analysis of the CSE447 Lab Project. The system is a secure web application integrating multiple cryptographic protocols as required by the course specification. All encryption algorithms have been implemented from scratch without relying on built-in framework encryption functions.

1.1 Project Overview

[Briefly describe the web application — its purpose, users, and core functionality.]

1.2 Technology Stack

[List programming language, framework, database, and any external libraries used (excluding built-in crypto).]

1.3 System Architecture Diagram

[Insert a system architecture / flow diagram here showing major modules and data flow.]

2. Login and Registration Module

The system provides secure registration and login flows. New users supply credentials which are validated, encrypted, and persisted. During login, stored encrypted data is retrieved and decrypted for verification.

2.1 Registration Flow

[Describe the step-by-step registration process (input validation → encryption → storage). Include a flowchart if possible.]

2.2 Login Flow

[Describe the login process (credential retrieval → decryption → comparison → session creation).]

2.3 Implementation Details

Requirement	Implementation Details
Login Module	<i>[Describe how users are authenticated — algorithm used, flow]</i>
Registration Module	<i>[Describe user registration, input fields, validation rules]</i>
Data Encrypted Before Storage	<i>[Specify which fields are encrypted and with which algorithm]</i>
Data Decrypted on Retrieval	<i>[Explain the decryption process on login/profile fetch]</i>

3. User Data Encryption and Decryption

All sensitive user information (e.g., username, email, contact info) is encrypted before storage using asymmetric encryption algorithms implemented from scratch, and decrypted upon retrieval.

3.1 Fields Encrypted

[List all database fields that are encrypted (e.g., name, email, phone). Specify which algorithm is applied to each.]

3.2 Encryption Algorithm - RSA Implementation

[Explain your from-scratch RSA implementation: key size, padding scheme, modular exponentiation approach, etc.]

3.3 Encryption Algorithm - ECC Implementation

[Explain your from-scratch ECC implementation: curve parameters (e.g., NIST P-256), point multiplication, encryption scheme (e.g., ECIES).]

3.4 How Both Algorithms Are Used Differently

[Clarify which part of the system uses RSA and which part uses ECC, satisfying the requirement that a single algorithm is not used for all operations.]

4. Password Hashing and Salting

Passwords are never stored in plaintext. A cryptographic hash function combined with a random salt is applied before storage to prevent dictionary and rainbow-table attacks.

4.1 Hashing Algorithm Used

[State the hash function used (e.g., SHA-256, SHA-3) and justify the choice.]

4.2 Salt Generation

[Describe how the random salt is generated, its length, and where it is stored relative to the hash.]

4.3 Verification Process

[Explain how a password is verified on login (retrieve salt → re-hash input → compare).]

5. Two-Factor Authentication (2FA)

The system enforces two-step verification: the user must pass both primary credential validation and a second authentication factor before a session is granted.

5.1 2FA Method

[Describe the second factor used (e.g., TOTP, email OTP, SMS code, security question). Explain the flow.]

5.2 Code Snippet

[Paste relevant 2FA implementation code here.]

6. Key Management Module

A dedicated Key Management Module handles the full lifecycle of cryptographic keys: secure storage, and rotation.

6.1 Key Storage Security

[Detail how private keys are protected in the database or file system (e.g., encrypted with another key, hardware token, etc.).]

6.2 Key Rotation Policy

[Describe the rotation schedule and how old keys are safely invalidated without breaking existing encrypted records.]

7. Post and Profile Management

Users can create, view, and edit posts, as well as view and update their profiles. All post and profile data is automatically encrypted before storage and decrypted on retrieval.

7.1 Post Module

[Describe the Create / Read / Update flow for posts. List all fields that are encrypted and the algorithm applied.]

7.2 Profile Module

[Describe the profile view / update flow. List which profile fields are encrypted.]

7.3 Screenshots

[Insert screenshots of the post creation, post listing, and profile management pages.]

8. Data Storage Security

All critical data - user information, posts, and cryptographic keys - is stored in encrypted form to prevent plaintext access even in the event of a database compromise.

8.1 Evidence of Encrypted Storage

[Insert a screenshot of the raw database records (showing ciphertext, not plaintext) to demonstrate encrypted storage.]

9. Message Authentication Code (MAC)

Message Authentication Codes (MACs) are used to verify the integrity of stored data and detect any unauthorized modifications. The system implements CBC-MAC or HMAC as required.

9.1 MAC Algorithm Used

[State whether CBC-MAC or HMAC is used. Justify the choice. Describe the implementation from scratch.]

9.2 Integrity Verification Flow

[Describe when and how MAC verification is performed (on every read, on critical operations, etc.).]

10. Role-Based Access Control (RBAC)

Role-Based Access Control defines distinct privilege levels for Administrators and Regular Users, ensuring that sensitive operations are restricted appropriately.

10.1 Roles Defined

[List all roles in the system (e.g., Admin, User) and their high-level responsibilities.]

10.2 Permission Matrix

Operation / Resource	Admin	Regular User
View own profile	✓	✓
Edit own profile	✓	✓
Create / Edit posts	✓	✓
Delete any post	✓	✗
View all user accounts	✓	✗
Manage / rotate keys	✓	✗
Assign roles to users	✓	✗
View audit logs	✓	✗

(Add or remove rows as appropriate for your system. Tick marks above are placeholders.)

11. Secure Session Management

Authentication tokens and session identifiers are managed securely to prevent session hijacking, fixation, and replay attacks.

11.1 Token Signing / Verification

[Describe how tokens are signed (e.g., HMAC-SHA256 or RSA) and how they are verified on each request.]

12. GitHub Repository and Project Structure

Field	Details
GitHub Repository URL	[https://github.com/your-username/your-repo-name]

12.1 Repository Structure

[Paste or describe the top-level folder/file structure of your repository (e.g., output of `tree` command or a directory listing).]

12.2 README Overview

[Summarise what your README.md contains: setup instructions, environment requirements, how to run the project locally.]

13. Conclusion

[Write a brief concluding paragraph reflecting on challenges faced, lessons learned, and the overall outcome of the project.]